E-Voting System

Project Report

By
Prajwal Vyavahare – AF05108439
Piyush Yadav – AF05102865

Index

Sr.no	Topic	Page no
1	Title of Project	1
2	Acknowledgement	3
3	Abstract	4
4	Introduction	5
5	System Analysis	8
6	System Design	15
7	Screenshots	18
8	Implementation	22
9	Results and Discussion	24
10	Conclusion	27
11	Future Scope	28
12	Bibliography and References	29

Acknowledgement

The project **"Event Management System"** is the Project work carried out by

Name	Enrollment No
Prajwal Vyavahare	**AF05108439**
Piyush Yadav	**AF05102865**

We are thankful to our project guide for guiding us to complete the Project.

Their suggestions and valuable information regarding the formation of the Project Report have provided us a lot of help in completing the Project and its related topics.

Abstract

The E-Voting System is a secure, efficient, and reliable digital platform designed to conduct elections electronically, replacing traditional paper-based voting methods. The system aims to enhance transparency, accuracy, and speed in the electoral process while minimizing human errors and fraudulent activities. It provides a user-friendly interface that allows registered voters to cast their votes remotely or through authorized systems while ensuring voter authentication and vote confidentiality.

The system is developed using modern technologies such as Java, Hibernate ORM, and MySQL, enabling seamless data management and secure storage of voter, candidate, and voting information. It incorporates key features such as voter registration, candidate management, vote casting, and result generation, all integrated into a centralized database-driven architecture.

By automating the voting process, the E-Voting System significantly reduces time consumption, operational costs, and manual effort involved in traditional elections. Additionally, it ensures data integrity and prevents duplication or manipulation of votes through secure authentication mechanisms. The system is scalable and can be further enhanced with advanced security features such as biometric verification and blockchain integration.

Overall, the E-Voting System provides a robust and efficient solution for modern electoral needs, contributing to fair, transparent, and accessible elections.

1. Introduction

The process of voting is a fundamental component of democratic systems, enabling citizens to express their choices and participate in governance. However, traditional voting methods—such as paper ballots or standalone electronic voting machines—often face challenges including long processing times, human errors, high operational costs, and security concerns. To overcome these limitations, modern technology offers a digital alternative in the form of an E-Voting System.

The E-Voting System is a computer-based application designed to conduct elections electronically with improved efficiency, accuracy, and transparency. It automates the entire voting process, from voter registration to result generation, ensuring secure handling of data and minimizing manual intervention.

.

1. Background of the Study

Voting systems have evolved significantly over time, from manual paper-based methods to electronic systems. Traditional methods often involve extensive manpower, time-consuming processes, and risks of data mismanagement. In large-scale elections, maintaining accuracy and transparency becomes even more challenging.

With the advancement of information technology, digital solutions have been introduced to simplify and enhance electoral processes. E-Voting systems aim to provide a centralized platform where voter data, candidate information, and voting records can be securely managed. These systems reduce dependency on physical infrastructure and enable faster, more reliable election management.

2. Motivation

The motivation behind developing this project arises from the need to address the inefficiencies and limitations of traditional voting methods. Some of the key challenges include: include:

- Manual errors in vote counting and record maintenance.
- Time-consuming voting and result declaration processes.
- High costs associated with printing, logistics and manpower.
- Risk of fraudulent activities and lack of transparency.
- Difficulty in accessing polling stations for remote or disabled voters

This project addresses these challenges by leveraging Java's object-oriented design and Hibernate's ORM capabilities to build a modular and scalable system that automates repetitive tasks and ensures smooth voting execution.

3. Significance of the Study

This project holds significance for multiple stakeholders:
For Voters : Provides a convenient and accessible way to cast votes with minimal effort.
For Election Authorities: Reduces administrative workload and improves election Management.
For Government/Organizations: Ensures transparent and reliable election results.
For Developers and Students: Serves as a practical example of integrating Java, Hibernate, and MySQL for real-world applications.

By digitizing the voting process, the system enhances accuracy, reduces operational costs, and increases trust in the electoral system.

4. Features of E-System:

The **E-Voting System** includes several key features to ensure smooth and secure election operations:

Voter Management

a) Register voters with details such as name, address, contact, and email.
b) Authenticate voters using login credentials or verification methods.
c) Prevent duplicate voting through unique voter identification.

Candidate Management

a) Add, update, and delete candidate information.
b) Display candidate details such as name, party and profile.

Voting Process

a) Allow voters to cast their vote securely
b) Ensure one vote per voter.
c) Maintain vote confidentiality and anonymity.

Admin Control

a) Manage voters, candidates, and elections.
b) Monitor voting activity and system status
c) Control election start and end times.

Security Features:

a) User authentication and authorization.
b) Data validation and encryption (optional enhancement).

c) Prevention of unauthorized access and manipulation.

5. Reporting and Analytics:

The E-Voting System provides reporting and analytical features to support decision-making and transparency:

- Generate election result reports instantly after voting ends.
- Display total votes received by each candidate.
- Provide summary reports of voter participation.
- Analyze voting trends and turnout percentages.
- Maintain historical records of past elections for reference.

These features help administrators evaluate election performance and ensure accountability in the voting process.

2. System Analysis

System analysis is a crucial phase in the Software Development Life Cycle (SDLC). It involves identifying the problems in the existing system, defining requirements, and evaluating the feasibility of the proposed solution. In the context of the E-Voting System, system analysis focuses on improving the traditional voting process by introducing a secure, efficient, and automated digital platform.

1. Problem Definition

Traditional voting systems face several challenges that affect efficiency, accuracy and transparency. These include:

- Time-Consuming Process : Manual voting and counting require significant time and manpower.
- Human Errors: Mistakes during vote counting or record maintenance can affect results.
- Lack of Transparency: Difficulty in verifying fairness and authenticity of the election process.
- High Operational Cost: Expenses for printing ballots, setting up polling stations, and staffing.
- Security Issues: Risk of vote tampering, duplication, or unauthorized access.
- Limited Accessibility: Voters in remote areas or with disabilities may face difficulties in voting.

These limitations highlight the need for a digital system that ensures secure, fast, and reliable voting.

2. Objectives of the System

The proposed E-Voting System aims to achieve the following objectives:

- Provide a secure and centralized platform for conducting elections.
- Enable voters to cast votes electronically with proper authentication.
- Maintain confidentiality and integrity of voting data.
- Automate vote counting and result generation for faster outcomes.
- Reduce manual effort and operational costs
- Improve transparency and trust in the election process.
- Support scalability for handling large numbers of voters and elections.

3. Feasibility Study

Feasibility analysis determines whether the proposed system is practical and achievable.

a. Technical Feasibility

The system is developed using technologies such as Java, Hibernate ORM, for database interaction MySQL. These tools are widely used, reliable, and capable of handling secure database operations.

Conclusion: The system is technically feasible.

b.Economic Feasibility

The project uses open-source technologies, reducing development and maintenance costs.

It also minimizes expenses related to paper ballots, manpower, and logistics.

Conclusion: The system is cost-effective and economically feasible.

c. Operational Feasibility

The system provides a user-friendly and structured interface for managing E-Voting operations. Users such as voters and administrators can easily adapt to it. It minimizes human errors and automates repetitive tasks.

Security features like controlled access ensure safe data handling and prevent unauthorized use.

Conclusion: The system is operationally feasible.

a. Time Feasibility

The project can be developed efficiently within the allotted timeframe using an iterative approach. Each module Voter, Candidate , Voting and Admin can be implemented and tested separately.

The modular structure enables continuous integration and early feedback.

Conclusion: The project is time-feasible.

4. System Requirements

a. Functional Requirements:

- The E-Voting System must support the following functionalities:

- Voter registration and authentication.
- Candidate registration and management.

- Admin login and system control.
- Secure vote casting.
- Prevention of duplicate voting.
- Vote storage and processing.
- Result calculation and display.
- Election creation and management.

b. Non-Functional Requirements:

- Usability: Simple console interface for easy interaction.
- Reliability: Ensure consistent data storage and retrieval using Hibernate.
- Scalability: System can handle multiple events and participants efficiently.
- Security: Prevent unauthorized access to sensitive voting data.
- Performance: CRUD operations and data retrieval should execute quickly..

c. Hardware Requirements:

- Processor: Intel i3 or higher.
- RAM: 4 GB minimum (8 GB recommended).
- Storage: 500 MB for database and reports.

d. Software Requirements

- Operating System: Windows / Linux / macOS
- Database: MySQL 8.0 or above
- Programming Language: Java 17 or higher
- Frameworks/Libraries: Hibernate ORM, Junit (for testing)
- IDE: IntelliJ IDEA / Eclipse / NetBeans
- Build Tool: Apache Maven

5. Proposed System

The proposed E-Voting System provides a centralized digital platform for conducting elections efficiently and securely.

Voter Management: Add and manage election candidates.

Election Management: Create and control election processes.

Vote Casting: Allow users to vote electronically with validation.

Result Generation: Automatically calculate and display results.

Data Centralization: Store all data in a structures database.

Security Measures: Prevent unauthorized access and duplicate voting.

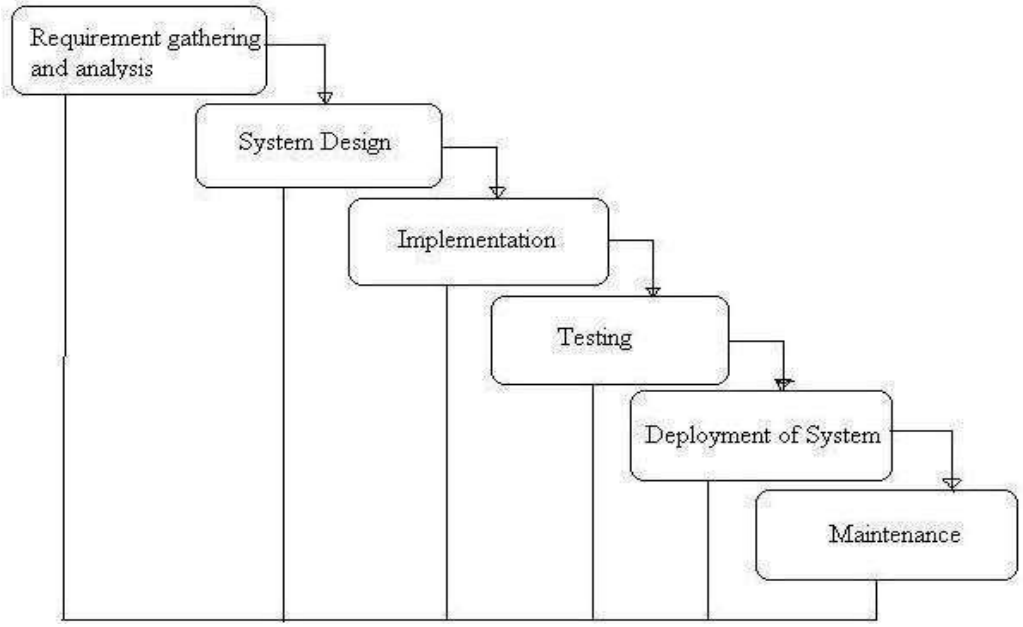


Fig. 5.2 . Phases of Development

6. Phases of Development

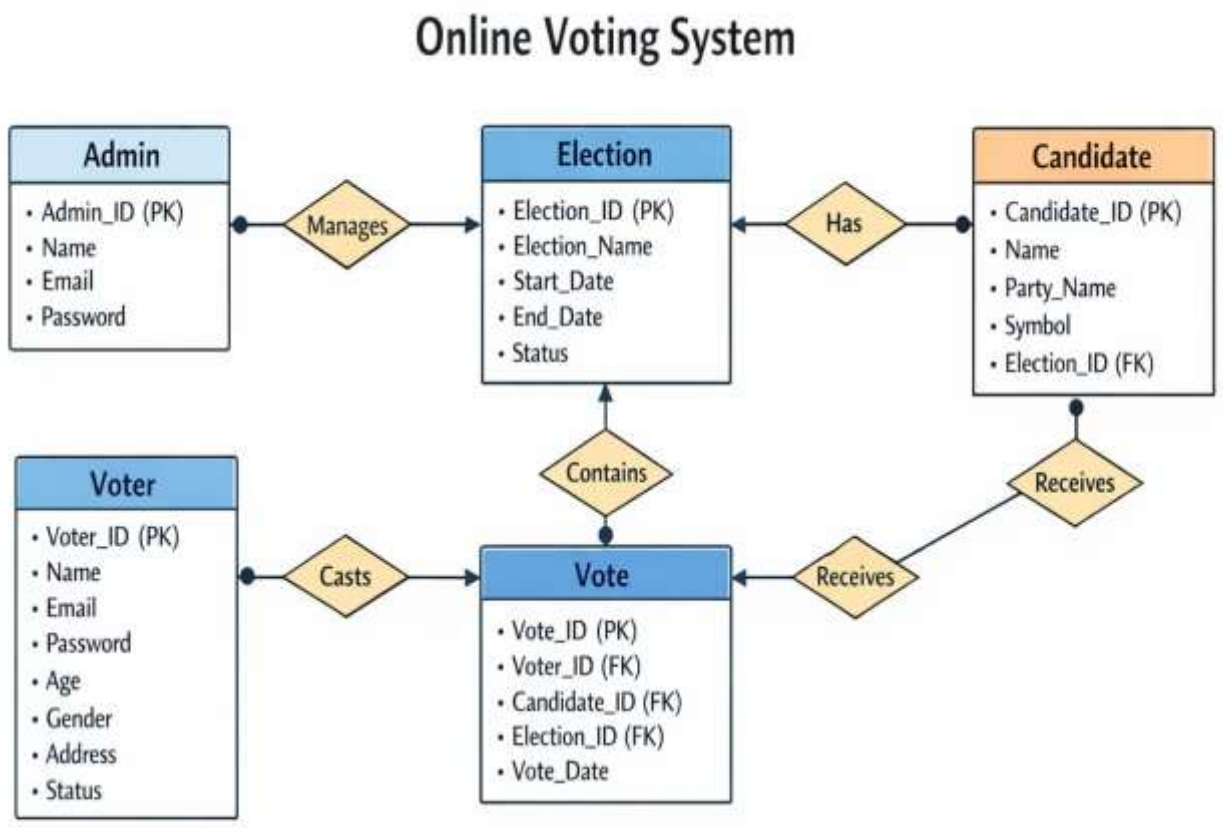
1. **Requirement Analysis & System Study**
   - Identify system needs and gather requirements.
   - Understand problems in traditional voting systems
2. **System Design**
   - Designing the database schema using MySQL to represent relationships among entities such as Voter, Candidate , Vote and Election.
   - Developing the architecture to ensure modularity and scalability.
   - Planning the console-based user interaction flow for efficient CRUD operations.
3. **Implementation (Coding)**
   - Backend development using Java with Hibernate ORM for database operations.

- Integration of MySQL for persistent data storage.

4. **Testing & Debugging**
   - Conducting unit testing on each module.
   - Performing integration testing to ensure smooth data flow across modules.
   - Debugging and refining system logic for improved reliability and accuracy.
5. **Deployment & Maintenance**
   - Deploying the application on local environments for usage and demonstration.
   - Ensuring proper configuration of the database and Hibernate settings.
   - Providing continuous maintenance for bug fixes, feature enhancements, and database optimization.

7. ER diagram:

Fig. 2.7. ER Diagram



The ER diagram for the Event Management System represents the logical structure of the database, showing entities, their attributes, and relationships among them. It provides a clear view of how admin, election, candidate, voter interact within the system.

1. **Entities and Their Attributes**

- Admin
  Attributes: Admin_Id (PK), Name, Email, Password.
  The admin is responsible for creating elections, managing candidates, monitoring voting, and ensuring system security
- Election
  Attributes: Election_Id (PK), Election_Name, Start_Date, End_Date, Status
  This entity controls the timeline and availability of voting. It links candidates and votes within a specific election.
- Candidate
  Attributes: Candidate_Id (PK), Name, Party_name, Symbol, Election_ID(FK)
  Each candidate belongs to one election and completes to receive votes from voters.
- Voter
  Attributes: Voter_Id (PK), Name, Email, Password, Age, Gender, Address, Status
  This entity ensures that only registered and eligible users can cast votes. It helps enforce the rule of one vote per voter.
- Vote
  Attributes: Vote_ID (PK), Voter_ID(FK), Candidate_ID(FK), Election_ID(FK), Vote_Data
  This entity acts as a bridge between Voter, Candidate, and Election, ensuring proper tracking of votes while maintaining integrity.

2. **Relationships**

- Admin → Election: One admin can manage multiple elections (1 : M).
- Election → Candidate: One election can have multiple candidates, but each candidate belongs to only one election. (1 : M).
- Election ↔ Vote: One election contains multiple votes, but each vote belongs to only one election. (1 : M) .
- Vote ↔Election: The Vote entity acts as a junction connecting all major entities.

(M : 1).

3. **ER Diagram Summary**

- Maintain data integrity and consistency.
- Ensure secure and valid voting.
- Prevent duplicate voting.
- Enable efficient vote counting and result generation.

3. System design

A) Modules

1. Admin Module

This module is responsible for managing the entire system.
They can:

- Add, view, update, and delete admin details.
- Create and manage elections.
- Add and manages candidates.
- Activate or deactivate elections.
- Monitor voting activities and results

2. Election Module

This module handles all election-related Operations.
Users can:

- Create new elections with details such as name, type, start date, and end date.
- Set election status (Upcoming, Active, Completed).
- Associate candidates with elections.
- Manage election timelines.

3. Voter Module

The voter module manages voter registration and participation.
Users can:

- Register voters with personal details.
- Authenticate voters using login credentials.
- Allow voters to view available elections.
- Enable vote casting.
- Restrict duplicate voting.

4. Candidate Module

This module manages candidates participating in elections.
Users can:

- Add, update and delete candidate details.
- Assign candidates to elections.
- Display candidate information to voters.

5. Voting Module

This module is responsible for the actual voting process.
Users can:

- Allow voters to cast votes.

- Store votes securely in the database.
- Ensure one vote per voter per election.
- Maintain vote confidentiality.

6. System Module (Backend Services)

This module manages internal operations and database interactions.
It includes:

- Performing CRUD operations on all entities.
- Manage database connections using Hibernate.
- Handle transactions and session management.
- Ensuring data consistency and integrity across all modules.

B) Data Structure of All Modules

We have organized one database named e-voting for the system design.
It is implemented using MySQL and consists of several customized tables representing the entities and relationships involved in the event management process.

1. Tables and Attributes

Admin Table (admin)
Stores information about administrators.
Attributes: admin_id, name, email, password.

Election Table (election)
Stores details of elections .
Attributes: election_id, election_name, start_date, end_date, status.

Candidate Table (candidate)
Stores information about candidates.
Attributes: candidate_id, name, party_name, symbol, election_id

Voter Table (voter)
Stores voters details.
Attributes: voter_id, name, email, password, age, gender, address, status

Vote Table (vote)
Stores all event details of vote.
Attributes: vote_id, voter_id, candidate_id, election_id, vote_date

2. Relationships Between Tables (ER/Class Diagram)

Admin → Election: One admin can manage multiple elections (1 : M).
Election → Candidate: One election has many candidates (1 : M).
Election ↔ Vote: One election contains many votes(1:M)
Voter↔ Vote: One voter casts one vote per election(1 : M)
Candidate↔ Vote: One candidate receives multiple votes(1 : M)

This structure ensures data normalization, eliminates redundancy, and maintains referential integrity across all modules. It supports scalable and efficient management of all voting-related data in the system.

4. Implementation

The implementation phase involves converting the system design into a functional software application. The E-Voting System was developed as a Java console-based application using Hibernate ORM for database management and MySQL as the relational database. This section describes the technologies, logic, and design used in the implementation.

1. Technology Stack

- Programming Language: Java (JDK 17)
- Framework: Hibernate ORM (for Object-Relational Mapping)
- Database: MySQL (Relational Database Management System)
- Build Tool: Apache Maven
- Testing Framework: Junit
- Logging: SLF4J (Simple Logging Façade for Java)
- Development Environment: IntelliJ IDEA / Eclipse

2. Backend Implementation

The backend handles all core logic and database operations using Hibernate. CRUD functionalities are implemented for each entity: Admin, Candidates, Election, Voter, and Vote.

Key Functionalities:

- Admin Management:
  Admin can log in using credentials and create, update elections.
- Election Management:
  Create and activate elections with name, date and status.
- Voter Management:
  Register voters with personal details.
- Candidate Management:
  Add candidate details such as name, party and symbol and assign candidates to specific elections.

3. Console Interface Implementation

Since this is a console-based system, a menu-driven interface allows the user to perform CRUD operations.

The user enters values, which are captured through Java's Scanner class and passed to DAO methods for execution.

4. Database Implementation

The database is implemented using MySQL with five main tables:

- Admin – Stores admin details.
- Voter – Stores voter information.
- Candidate – Stores candidate details
- Election – Stores election information
- Vote – Stores all votes

Relationships:

- One Admin → Many Elections
- One Election → Many Candidates
- One Election → Many Votes
- One Voter → One vote per election
- One candidate → Many votes

Security Implementation

- Foreign Key Constraints: Maintain relational integrity between entities.
- Transaction Management: All CRUD operations are wrapped in Hibernate transactions to prevent partial commits.
- Exception Handling: Ensures the system gracefully handles invalid inputs or connection failures.
- Data Validation: Basic checks (e.g., null values, invalid IDs) are performed during input and database operations.

5. Results and Discussion

The development and implementation of the E-Voting System produced several significant outcomes in terms of functionality, performance, and usability. This section presents the results achieved after testing the system and discusses its effectiveness, challenges, and limitations.

---

1. Results

a. Functional Results

The system successfully performs all the core functionalities required for conducting an election:

- **Admin Module:**
  Admin can create and manage elections, add candidates, and control the voting process. Election activation and deactivation work correctly.
- **Voter Module:**
  Voters can register, log in securely, and cast their votes. The system ensures that each voter can vote only once per election.
- **Candidate Module:**
  Candidate details can be added, updated, and deleted. Candidates are correctly linked to specific elections.
- **Voting Process:**
  Votes are recorded accurately in the database with proper linkage to voter, candidate, and election. The system prevents duplicate voting.
- **Result Generation:**
  Votes are counted automatically, and results are displayed instantly after the election ends.
- **Database Operations:**
  All CRUD (Create, Read, Update, Delete) operations are executed successfully using Hibernate, ensuring data persistence and consistency.

---

b. Non-Functional Results

- **Usability:**
  The system provides a simple and user-friendly interface, allowing users to navigate easily through different modules.
- **Performance:**
  The system processes votes quickly and generates results efficiently, even with multiple users.

- **Reliability:**
  Proper exception handling ensures that the system performs consistently without crashes.
- **Security:**
  Authentication mechanisms and database constraints ensure secure voting and prevent unauthorized access.
- **Data Integrity:**
  Relationships between entities (Voter, Candidate, Election, Vote) are maintained correctly using foreign keys.
- **Scalability:**
  The modular structure allows the system to handle larger datasets and additional features in the future.

---

2. Discussion

1. Achievement of Objectives

- The system successfully meets its primary objectives:
  - Secure vote casting
  - Prevention of duplicate voting
  - Automated vote counting
  - Efficient election management
- The integration of different modules ensures smooth workflow and data consistency.

---

2. User Experience

- The system provides a structured and intuitive experience for both voters and administrators.
- Clear navigation and simple operations make it easy to use, even for beginners.

---

3. Challenges Faced

During development and implementation, several challenges were encountered:

- Managing relationships between entities (especially Vote, Voter, and Candidate)
- Ensuring strict enforcement of the “one vote per voter” rule
- Handling authentication and validation processes
- Maintaining data consistency across multiple modules
- Debugging database connectivity and Hibernate configurations

---

4. Limitations

- The system is currently **console-based or basic UI**, lacking advanced graphical interface
- Limited security features (no biometric or multi-factor authentication)
- No real-time analytics dashboard
- Internet dependency for online voting
- No integration with external identity verification systems

---

5. Future Enhancements (Discussion Insight)

- Introduce **biometric authentication** for stronger security
- Implement **blockchain technology** for tamper-proof voting
- Develop a **web/mobile interface** for better accessibility
- Add **real-time analytics and dashboards**
- Integrate **cloud deployment** for scalability

6. Conclusion

The development of the E-Voting System successfully demonstrates the design and implementation of a secure, efficient, and reliable platform for conducting elections digitally. The system addresses the major limitations of traditional voting methods by introducing automation, accuracy, and improved transparency in the electoral process.

The E-Voting System enables users to:

- Register and authenticate voters securely
- Manage candidates and elections efficiently
- Cast votes electronically with confidentiality
- Ensure one vote per voter to prevent duplication
- Generate accurate and instant election results

One of the key achievements of this system is the integration of multiple modules such as Voter, Candidate, Election, and Vote into a centralized database structure. This ensures smooth coordination, data consistency, and reliable performance. The use of technologies like Java, Hibernate ORM, and MySQL further enhances the system's efficiency and scalability.

Additionally, the system reduces manual effort, minimizes human errors, and lowers operational costs associated with traditional voting systems. The automated vote counting mechanism ensures quick and error-free result generation, improving overall election management.

Although the current system is functional and effective, it can be further enhanced with advanced features such as biometric authentication, blockchain integration, and a web or mobile interface to increase security and accessibility.

In conclusion, the E-Voting System provides a modern and practical solution for digital elections, contributing to a more transparent, secure, and user-friendly voting process. It lays a strong foundation for future advancements and real-world implementation in various domains such as educational institutions, organizations, and government elections.

7. Future scope

Although the E-Voting System successfully fulfills its current objectives, there are several opportunities for enhancement to make the system more secure, scalable, user-friendly, and suitable for real-world deployment. The following improvements can be implemented in future versions:

---

1. Web-Based and Mobile Application

- Develop a **fully functional web application** using frameworks like Spring Boot or React.
- Create a **mobile application (Android/iOS)** to allow users to vote from anywhere.
- Provide a responsive and user-friendly interface for better accessibility.

Benefit:
Improves usability and allows remote participation in elections.

---

2. Biometric Authentication

- Integrate biometric verification methods such as:
  - Fingerprint recognition
  - Face recognition
- Link voter identity with biometric data for secure authentication.

Benefit:
Prevents unauthorized access and ensures that only genuine voters can cast votes.

---

3. Blockchain Integration

- Implement blockchain technology to store votes securely.
- Ensure that once a vote is recorded, it cannot be altered or deleted.
- Provide a transparent and tamper-proof voting system.

Benefit:
Enhances trust, security, and transparency in the voting process.

---

4. Advanced Security Mechanisms

- Implement **multi-factor authentication (MFA)**
- Use **data encryption techniques** for secure vote storage

- Apply **secure communication protocols (HTTPS, SSL)**
- Introduce **audit logs** to track system activities

Benefit:
Strengthens system security and protects against cyber threats.

---

5. Real-Time Monitoring and Analytics

- Provide real-time dashboards showing:
  - Voter turnout
  - Voting trends
  - Election progress
- Use analytics tools to generate insights and reports

Benefit:
Helps administrators make informed decisions and improves transparency.

---

6. Role-Based Access Control

- Introduce different user roles such as:
  - Admin
  - Election Officer
  - Voter
- Assign permissions based on roles

Benefit:
Improves system security and management by restricting access.

---

7. Multi-Language Support

- Provide support for multiple languages
- Enable users to select their preferred language

Benefit:
Makes the system accessible to a wider audience.

---

8. Integration with Government Databases

- Connect the system with official identity databases (e.g., Aadhaar or national ID systems)
- Automate voter verification

Benefit:
Ensures authenticity and eliminates fake registrations.

---

9. Offline Voting Support

- Develop a hybrid system that supports both online and offline voting
- Sync offline votes with the central database

Benefit:
Ensures voting accessibility even in areas with limited internet connectivity.

---

10. Cloud Deployment

- Deploy the system on cloud platforms like AWS or Azure
- Ensure high availability and scalability

Benefit:
Allows the system to handle large-scale elections efficiently.

---

11. Automated Notifications

- Send notifications via:
  - Email
  - SMS
- Inform users about:
  - Election dates
  - Voting reminders
  - Results

Benefit:
Improves user engagement and participation.

---

12. Graphical Reports and Dashboards

- Generate visual reports such as:
  - Bar charts
  - Pie charts
- Show voting statistics and results clearly

Benefit:
Makes data easier to understand and analyze.

8. Bibliography and References

In preparing this project report on the **E-Voting System**, various books, research papers, and online resources were referred to. These sources provided valuable knowledge about system design, database management, software development methodologies, and secure voting systems..

Books

1. Rajaraman, V. – *Fundamentals of Computers*, Prentice Hall of India.
2. Abraham Silberschatz, Henry Korth, S. Sudarshan – *Database System Concepts*, McGraw Hill.
3. Roger S. Pressman – *Software Engineering: A Practitioner's Approach*, McGraw Hill.
4. Ian Sommerville – *Software Engineering*, Pearson Education.
5. Herbert Schildt – *Java: The Complete Reference*, McGraw Hill Education.

Research Papers / Journals

- **International Journal of Computer Applications (IJCA)** – Research on E-Voting Systems and secure digital voting mechanisms.
- **IEEE Xplore Digital Library** – Articles on online voting security, authentication, and system design.
- **Springer Journal of Information Systems** – Studies on database-driven applications and secure systems.

- Research papers on **Blockchain-based Voting Systems** and **Secure Authentication Techniques**.

Websites

1. https://hibernate.org/ - Official Hibernate ORM documentation.
2. https://www.mysql.com/ - MySQL official documentation.
3. https://maven.apache.org/ - Apache Maven project management tool documentation.
4. https://docs.oracle.com/javase/ - Official Java SE documentation.
5. https://www.baeldung.com/hibernate-tutorial - Tutorials and best practices for Hibernate in Java.